**Functions — 80/20 Interview Notes for Machine Learning**

This chapter is foundational because **models, loss functions, activation functions, optimization objectives, and neural-network layers are all functions**. For interviews, focus on the concepts that let you reason about a function mathematically **and** implement it cleanly.

---

**1. What is a Function?**

**Definition**

A function is a mapping from a set $X$to a set $Y$:

$$f: X \to Y$$

such that **each input** $x \in X$**maps to at most one output** $y \in Y$.

We write:

$$y = f(x)$$

**Mental model**

f

X ----------> Y

x ----------> f(x)

For example:

$$f(x) = x^2$$

Input X Output Y

-2 ——————————> 4

-1 ——————————> 1

0 ——————————> 0

1 ——————————> 1

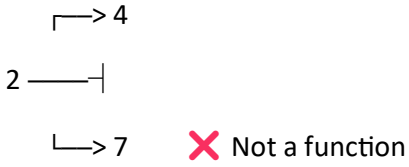
2 ——————————> 4

Notice:

Multiple inputs may produce the same output.

That's allowed.

But one input cannot produce two different outputs.




❌ Not a function

**Interview question**

**Q: What makes something a function?**

**Answer:**

Every input in the domain must have at most one corresponding output. Different inputs may map to the same output, but a single input cannot map to multiple outputs.

---

**2. Domain, Codomain and Image**

Suppose:

$$f: X \to Y$$

There are three related concepts.

**Domain**

The set of valid inputs.

$$\mathrm{dom}(f)$$

**Codomain**

The target set $Y$.

**Image / Range**

The outputs that are **actually produced** by the function.

$$\mathrm{im}(f) = \{f(x): x \in X\}$$

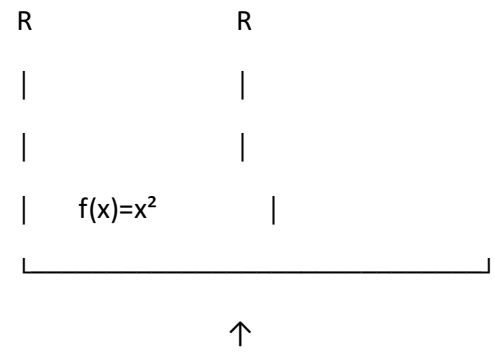
**Example**

Consider:

$$f: \mathbb{R} \to \mathbb{R}$$
$$f(x) = x^2$$

Then:

Domain Codomain




Actual outputs

= [0,∞)

Therefore:

$$\text{dom}(f) = \mathbb{R}$$

but

$$\text{im}(f) = [0, \infty)$$

**Important interview distinction**

For:

$$f: \mathbb{R} \to \mathbb{R},\ f(x) = x^2$$

the **codomain is** $\mathbb{R}$, but the **image is** $[0, \infty)$.

This distinction matters for understanding **surjectivity**.

---

**3. Injective, Surjective and Bijective**

This is one of the highest-value theoretical concepts from this chapter.

---

**3.1 Injective**

A function is injective if:

Different inputs always produce different outputs.

Formally:

$$f(x_1) = f(x_2) \Rightarrow x_1 = x_2$$

**Diagram**

X Y

a ——————————> 1

b ——————————> 2

c ——————————> 3

No two inputs point to the same output.

**Example**

$$f(x) = 2x + 1$$

is injective over $\mathbb{R}$.

Why?

$$2x_1 + 1 = 2x_2 + 1$$

implies

$$x_1 = x_2$$

**ML connection**

Many transformations need to preserve enough information to distinguish inputs. Injectivity becomes relevant when thinking about **invertible transformations**, normalizing flows, and information-preserving mappings.

---

**4. Surjective**

A function is surjective if:

Every element of the codomain is reached by at least one input.

Formally:

$$\forall y \in Y,\ \exists x \in X: f(x) = y$$

**Diagram**

X Y

a ——————————> 1

b ——————————> 2

c ——————————> 3

↑

|

every output

is reached

**Example**

$$f(x) = x^3$$

where:

$$f: \mathbb{R} \to \mathbb{R}$$

is surjective.

Why?

For every $y \in \mathbb{R}$:

$$x = \sqrt[3]{y}$$

exists.

Therefore:

$$f(x) = y$$

**5. Bijective**

A function is bijective when it is:

Injective + Surjective

Diagram:

X Y

a ——————————> 1

b ——————————> 2

c ——————————> 3

Exactly one input corresponds to each output.

Therefore the function can be reversed.

**Example**

$$f(x) = 2x + 1$$

from:

$$\mathbb{R} \to \mathbb{R}$$

is bijective.

Its inverse is:

$$f^{-1}(y) = \frac{y-1}{2}$$

**Critical relationship**

Injective → no two inputs share output

Surjective → every codomain element is reached

Bijective → both

Bijective → inverse exists

**Interview question**

**Q: When does a function have an inverse?**

For the function to have a proper inverse between its domain and codomain, it must be **bijective**.

---

**6. Why $x^2$ Is Not Invertible on $\mathbb{R}$**

Consider:

$$f(x) = x^2$$

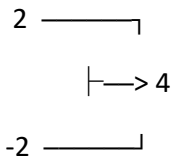
We have:

$$f(2) = 4$$

and

$$f(-2) = 4$$

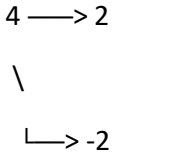
Therefore:




Two inputs produce the same output.

Therefore it is **not injective**.

If we try to reverse the arrows:




then 4 has two outputs.

That violates the definition of a function.

**Important fix**

Restrict the domain:

$$f: [0, \infty) \to [0, \infty)$$
$$f(x) = x^2$$

Now it becomes bijective.

Its inverse is:

$$f^{-1}(x) = \sqrt{x}$$

---

**7. Function Composition** ⭐ ⭐ ⭐

This is extremely important for ML.

Suppose:

$$g: X \to Y$$

and

$$f: Y \to Z$$

Then:

$$f \circ g: X \to Z$$

and:

$$(f \circ g)(x) = f\big(g(x)\big)$$

**Key rule**

$$\boxed{g \text{ happens first}}$$

because:

$$f\big(g(x)\big)$$

means:

x

|

▼

g

|

▼

g(x)

|

▼

f

|

▼

f(g(x))

---

**8. Logistic Regression as Function Composition**

This is an important ML example.

Linear function:

$$l(x) = ax + b$$

Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Logistic regression:

$$f(x) = \sigma(ax + b)$$

Therefore:

$$\boxed{f = \sigma \circ l}$$

Diagram:

x

|

▼

Linear transformation

ax + b

|

▼

Sigmoid

1/(1+e^-x)

|

▼

Probability

This is the mathematical foundation for understanding neural-network layers.

---

**9. Neural Networks = Function Composition** ⭐ ⭐ ⭐

Suppose a neural network has:

$$f_1, f_2, f_3$$

Then:

$$f(x) = f_3\left(f_2\left(f_1(x)\right)\right)$$

Diagram:

Input

|

▼

Layer 1

|

▼

Layer 2

|

▼

Layer 3

|

▼

Output

Mathematically:

$$f = f_3 \circ f_2 \circ f_1$$

This is one of the most important mental models in ML mathematics.

A neural network is a composition of parameterized transformations.

---

**10. Why Composition Matters for Deep Learning**

Consider:

$$f(x) = W_3\sigma(W_2\sigma(W_1x + b_1) + b_2) + b_3$$

This looks complicated.

But conceptually:

x

|

▼

Linear transformation

$W_1x + b_1$

|

▼

Activation

σ(...)

|

▼

Linear transformation

$W_2(...)$

|

▼

Activation

σ(...)

|

▼

Linear transformation

$W_3(...)$

|

▼

Output

So the complexity of a neural network comes largely from **composing many simple functions**.

Later, calculus gives us the chain rule for differentiating this composition.

---

**11. Function Addition and Scalar Multiplication**

Given:

$$f: X \to \mathbb{R}$$

and:

$$g: X \to \mathbb{R}$$

we can define:

$$(f + g)(x) = f(x) + g(x)$$

and:

$$(cf)(x) = cf(x)$$

**Example**

Let:

$$f(x) = x^2$$
$$g(x) = x + 1$$

Then:

$$(f + g)(x) = x^2 + x + 1$$

For $x = 2$:

$$f(2) + g(2) = 4 + 3 = 7$$

---

**12. Functions as Transformations**

This is a better mental model for modern ML than simply thinking about curves.

Instead of:

f(x) = formula

think:

Input space

|

| transformation f

▼

Output space

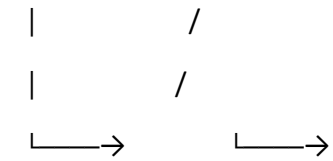
For example:

$$f: \mathbb{R}^2 \to \mathbb{R}^2$$

might rotate vectors.

Before After

↑ ↗



**Neural-network interpretation**

A layer transforms the representation:

$$x \to h_1 \to h_2 \to h_3$$

Each layer changes the representation of the data.

This is particularly useful when thinking about **embeddings and representation learning**.

---

**13. Functions in Python**

The mathematical:

$$f(x) = x^2$$

becomes:

```
def square(x):
   return x ** 2
```

Usage:

```
square(5)
```

Output:

25

**Important concept**

Python functions are **first-class objects**.

That means functions can be:

- stored in variables
- passed to other functions
- returned from functions
- stored in collections

This becomes important for ML frameworks and functional-style programming.

---

**14. Functions as Arguments**

Example:

```
def apply_function(f, x):
    return f(x)


def square(x):
    return x ** 2


apply_function(square, 5)
```

Result:

25

Diagram:

```
square
  |
  | passed as argument
  ▼
apply_function
  |
  ▼
square(5)
  |
  ▼
25
```

**Interview question**

**Q: What does "first-class function" mean?**

A function can be treated like any other value: it can be assigned to variables, passed as an argument, returned from another function, and stored in data structures.

**15. Implementing Function Composition**

Mathematical definition:

$$(f \circ g)(x) = f\big(g(x)\big)$$

Python:

```
def compose(f, g):
    def composition(x):
        return f(g(x))

    return composition
```

Example:

```
def square(x):
    return x ** 2

def cube(x):
    return x ** 3

square_after_cube = compose(square, cube)

square_after_cube(2)
```

Calculation:

$$2^3 = 8$$

then:

$$8^2 = 64$$

Result:

64

**Interview point**

Do not confuse:

compose(square, cube)

with:

square(cube(x))

The first **creates a new function**.

The second **executes the functions immediately**.

---

**16. Parameterized Functions ⭐ ⭐ ⭐**

This is extremely important for ML.

Consider:

$$f(x) = ax + b$$

Here:

- $x$= input
- $a, b$= parameters

A naive implementation:

```
def linear(x, a, b):
    return a * x + b
```

But ML models can contain thousands or millions of parameters.

Passing them individually becomes impractical.

---

**17. Callable Objects**

Python allows objects to behave like functions using:

__call__()

Example:

```
class Linear:
    def __init__(self, a, b):
```

```
        self.a = a
        self.b = b

    def __call__(self, x):
        return self.a * x + self.b
```

Now:

```
f = Linear(2, -1)
```

```
f(2)
```

returns:

3

because:

$$2(2) - 1 = 3$$

**Why is this useful?**

The object stores the parameters:

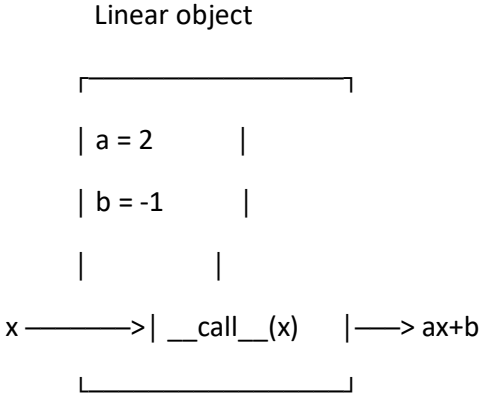
f.a

f.b

while still behaving like a function:

f(x)

So we get:



---

**18. Why Callable Objects Matter in ML**

This is the key idea:

Mathematical model

f(x; θ)

where:

$$\theta$$

represents model parameters.

For example:

$$f(x) = ax + b$$

can be represented as:

model = Linear(a=2, b=-1)

Now:

model(x)

performs inference.

The parameters remain attached to the model.

This is conceptually similar to how ML frameworks represent layers/models.

---

**19. Parameters**

A simple implementation:

```
class Linear:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def __call__(self, x):
```

```
        return self.a * x + self.b


    def parameters(self):
        return {
            "a": self.a,
            "b": self.b
        }
```

Usage:

```
model = Linear(2, -1)


model.parameters()
```

Output:

```
{
    "a": 2,
    "b": -1
}
```

**Why important?**

Training algorithms need access to model parameters.

Conceptually:

```
Model
 |
 ├── parameters
 |      ├── W
 |      ├── b
 |      └── ...
 |
 └── forward(x)
```

Later:

parameters

|

▼

forward

|

▼

loss

|

▼

gradient

|

▼

update parameters

That is the core training loop.

---

**20. __repr__() and Debugging**

Instead of:

<__main__.Linear object at 0x...>

implement:

```
def __repr__(self):
    return f"Linear(a={self.a}, b={self.b})"
```

Then:

model

shows:

Linear(a=2, b=-1)

**Why should an ML engineer care?**

Because debugging production/training systems often requires inspecting:

- model configuration

- layer parameters
- hyperparameters
- object state

Good representations reduce debugging time.

---

**21. A Function Base Class**

A simple abstraction:

```
class Function:
    def __call__(self, *args, **kwargs):
        raise NotImplementedError


    def parameters(self):
        return {}
```

Then:

```
class Sigmoid(Function):
    def __call__(self, x):
        return 1 / (1 + np.exp(-x))
```

Usage:

```
sigmoid = Sigmoid()


sigmoid(2)
```

Approximately:

$$0.8808$$

---

**22. Why Use a Base Class?**

Without a common interface:

Linear

Sigmoid

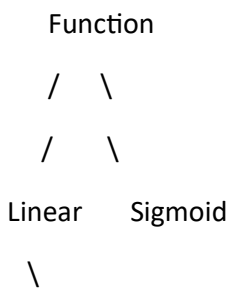
ReLU

Composition

...

may all behave differently.

With a common interface:




they can all support:

f(x)

and:

f.parameters()

This is an example of **polymorphism**.

**Interview connection**

This is the same engineering idea used in larger ML systems:

Different components implement a common interface, allowing them to be composed without the caller needing to know their concrete type.

---

**23. Composition as an Object**

A plain Python composition function returns a normal Python function.

But that loses the Function interface.

Instead:

```
class Composition(Function):
    def __init__(self, *functions):
        self.functions = functions
```

```
    def __call__(self, x):
        for f in reversed(self.functions):
            x = f(x)

        return x
```

Example:

```
model = Composition(
    Linear(2, -1),
    Sigmoid()
)
```

Then:

```
model(2)
```

works.

And:

```
isinstance(model, Function)
```

returns:

True

---

**24. Why Reverse the Functions?**

Suppose:

```
Composition(f, g)
```

Mathematically:

$$f \circ g$$

means:

$$f\big(g(x)\big)$$

So:

x

|

▼

g

|

▼

f

|

▼

output

But if the functions are stored as:

[f, g]

we must process:

reversed(functions)

so that:

g → f

is executed.

**Classic interview trap**

$$f \circ g \neq g \circ f$$

in general.

Function composition is **not commutative**.

---

**25. Logistic Regression — Complete Example**

Let's combine the concepts.

import numpy as np

```
class Linear:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def __call__(self, x):
        return self.a * x + self.b


class Sigmoid:
    def __call__(self, x):
        return 1 / (1 + np.exp(-x))


linear = Linear(2, -1)
sigmoid = Sigmoid()


def logistic_model(x):
    return sigmoid(linear(x))
```

For:

logistic_model(2)

first:

$$2(2) - 1 = 3$$

then:

$$\sigma(3) = \frac{1}{1 + e^{-3}}$$

approximately:

$$0.9526$$

So:

x = 2

|

▼

2x - 1

|

▼

3

|

▼

sigmoid

|

▼

0.9526

---

**26. Functions and Machine Learning Training** ⭐ ⭐ ⭐

The most important connection to remember:

A model is:

$$\hat{y} = f(x; \theta)$$

where:

- $x$= input
- $\theta$= parameters
- $f$= model/function
- $\hat{y}$= prediction

Then define a loss:

$$L(\theta)$$

Training means finding parameters that minimize the loss:

$$\theta^* = \arg\min_{\theta} L(\theta)$$

For linear regression:

$$\hat{y} = ax + b$$

and:

$$L(a, b) = \frac{1}{n}\sum_{i=1}^{n}(ax_i + b - y_i)^2$$

So ML training is fundamentally a **function optimization problem**.

---

**27. The Big Picture**

You should be able to see the entire chain:

Input x

|

▼

Model function f(x; θ)

|

▼

Prediction ŷ

|

▼

Loss function L(θ)

|

▼

Optimization

|

▼

Better parameters θ

Later calculus enters:

Loss

|

▼

Derivative / Gradient

|

▼

Direction of improvement

|

▼

Gradient Descent

|

▼

Updated parameters

So this chapter is preparing you for **derivatives, gradients, chain rule, and backpropagation**.

---

**28. Functions vs Methods vs Callable Objects**

For interviews, distinguish these.

**Normal function**

```
def square(x):
    return x * x
```

**Method**

```
class Model:
    def predict(self, x):
```

...

Called as:

model.predict(x)

**Callable object**

class Square:

  def __call__(self, x):

    return x * x

Called as:

square = Square()

square(x)

because Python translates:

square(x)

conceptually into:

square.__call__(x)

---

**29. High-Value 80/20 Interview Questions**

**Q1. What is a function?**

A mapping from a domain to a codomain where each input has at most one output.

---

**Q2. Can two inputs have the same output?**

Yes.

Example:

$$f(x) = x^2$$

because:

$$f(2) = f(-2) = 4$$

This means the function is not injective.

---

**Q3. What is injectivity?**

Different inputs cannot map to the same output:

$$f(x_1) = f(x_2) \Rightarrow x_1 = x_2$$

---

**Q4. What is surjectivity?**

Every element of the codomain has at least one preimage.

---

**Q5. What is bijectivity?**

A function that is both injective and surjective.

---

**Q6. Why is bijection important?**

A bijection has a well-defined inverse.

---

**Q7. What is function composition?**

$$(f \circ g)(x) = f\big(g(x)\big)$$

$g$executes first.

---

**Q8. Is function composition commutative?**

Generally, no.

$$f \circ g \neq g \circ f$$

---

**Q9. How is logistic regression related to composition?**

$$f(x) = \sigma(ax + b)$$

is:

$$f = \sigma \circ l$$

where:

$$l(x) = ax + b$$

---

**Q10. How is a neural network related to composition?**

A neural network is a composition of parameterized transformations:

$$f(x) = f_n\left(\cdots f_2\big(f_1(x)\big)\right)$$

---

**Q11. What is a first-class function in Python?**

A function that can be passed around like a value.

---

**Q12. What does __call__() do?**

It allows an object to be called like a function.

obj(x)

invokes:

obj.__call__(x)

---

**Q13. Why use callable objects for ML models?**

They allow us to combine:

function behavior

+

parameter/state storage

inside one object.

---

**Q14. Why use a common Function base class?**

To establish a consistent interface and enable polymorphic composition.

---

**Q15. Why is function composition important for neural networks?**

Because each layer transforms the output of the previous layer:

$$x \to f_1(x) \to f_2\big(f_1(x)\big) \to \cdots$$

---

**30. Implementation You Should Actually Practice**

Don't implement every piece from the chapter. For the **80/20 interview value**, implement these yourself.

**Exercise 1 — Basic function**

```
def square(x):
    return x ** 2
```

**Exercise 2 — Function as argument**

```
def apply(f, x):
    return f(x)
```

**Exercise 3 — Composition**

```
def compose(f, g):
    def composed(x):
        return f(g(x))


    return composed
```

**Exercise 4 — Callable parameterized model**

```
class Linear:
    def __init__(self, a, b):
        self.a = a
        self.b = b


    def __call__(self, x):
        return self.a * x + self.b
```

**Exercise 5 — Function abstraction**

```
class Function:
    def __call__(self, *args, **kwargs):
        raise NotImplementedError
```

```
    def parameters(self):
        return {}
```

**Exercise 6 — ML composition**

Implement:

Linear → Sigmoid

so that:

model(x)

produces the logistic-regression prediction.

---

**31. What You DON'T Need to Overstudy**

Using the 80/20 principle, don't spend excessive time on:

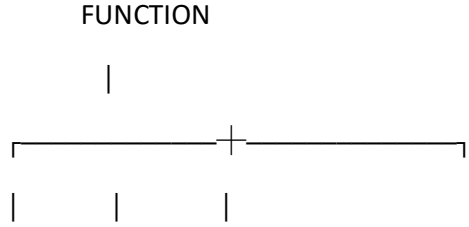
- the set-theoretic construction of functions as subsets of $X \times Y$
- philosophical discussions about what a function "really is"
- exotic function spaces
- highly formal proofs of basic properties
- advanced category-theory interpretations
- implementing every possible function operation
- Python __repr__ details beyond knowing why useful debugging representations matter

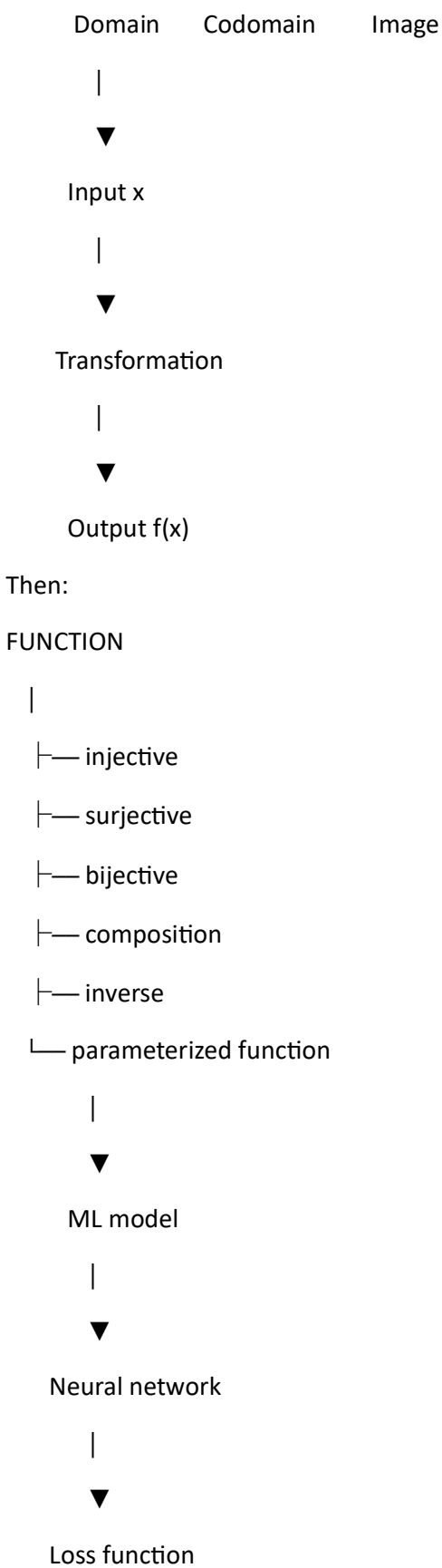
Understand the definitions, but prioritize **using them**.

---

**32. Your Mental Model for This Chapter**

Remember this:




Domain
Codomain
Image
Input x
Transformation
Output f(x)

Then:


FUNCTION
injective
surjective
bijective
composition
inverse
parameterized function
ML model
Neural network
Loss function

|

▼

Optimization

|

▼

Derivatives

|

▼

Gradient descent

|

▼

Backpropagation

**The 5 things I would make you master**

If your goal is **ML/AI engineering interviews**, be able to explain and implement these without notes:

1. **Function → domain/codomain/image**
2. **Injective / surjective / bijective → inverse**
3. **Composition → logistic regression → neural networks**
4. **First-class functions + __call__() → parameterized ML models**
5. **Model function → loss function → optimization → derivatives**

That is the core 80/20 of this chapter.